

Lab 1:

Query Query History

```
1 EXPLAIN (ANALYZE , BUFFERS )
2 SELECT c.first_name, c.last_name, r.rental_date
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN	
	text
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=6.801..23.577 rows=32.00 loops=1)
2	Buffers: shared read=153
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=1.027..1.034 rows=1.00 loo...
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared read=3
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=5.763..22.412 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared read=150
11	Planning:
12	Buffers: shared hit=187 read=13 dirtied=2
13	Planning Time: 58.162 ms
14	Execution Time: 23.838 ms

1. Identify the Roles

- **Outer Table (Driving Table - top node):** `customer` (accessed via `Index Scan using customer_pkey`).
- **Inner Table (Driven Table - bottom node):** `rental` (accessed via `Seq Scan on rental r`).

2. Why Nested Loop?

- **Rows returned by `customer`:** Exactly **1 row** (`rows=1.00`), because the query filters directly on the primary key condition `customer_id = 1`.
- **Why Hash Join is "too expensive":**
 - A Hash Join requires an upfront "Build Phase" to allocate memory (`work_mem`), run a hash function, and construct an in-memory hash table.
 - Constructing and managing a hash table just to look up **a single record** introduces unnecessary overhead. The Optimizer evaluates that running a single direct iteration via Nested Loop has a much lower startup and total cost.

3. Efficiency (**loops = 1** on the Inner Table node)

- In a Nested Loop join, the inner relation is executed once for every row returned by the outer relation (N loops for N outer rows).
- Because the outer table (**customer**) returned only **1 row**, the inner scan on **rental** was executed **exactly once (loops=1)**, avoiding repeated scans across the table.

Lab 2:

Query

Query History

1

-- Run this and look for "Hash Join " and "Hash" nodes

2

EXPLAIN (ANALYZE, BUFFERS)

Data Output

Messages

Notifications



















SQL

Showing rows: 1 to 14





Page No: 1

of 1









QUERY PLAN

text

1

Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=3.542..7.531 rows=1000.00 loops=1)

2

Hash Cond: (f.language_id = l.language_id)

3

Buffers: shared hit=88 read=1

4

-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.084..2.112 rows=1000.00 loops=1)

5

Buffers: shared hit=88

6

-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=3.412..3.416 rows=6.00 loops=1)

7

Buckets: 1024 Batches: 1 Memory Usage: 9kB

8

Buffers: shared read=1

9

-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=3.372..3.378 rows=6.00 loops=1)

10

Buffers: shared read=1

11

Planning:

12

Buffers: shared hit=276 read=7 dirtied=1

13

Planning Time: 70.109 ms

14

Execution Time: 7.970 ms

Total rows: 14

Query complete 00:00:00.424

CRLF

Ln 1, Col 1

1. Identify Phases

- **Hashed (Built) Table: language** (Inner Table). It is scanned once under the **Hash** node to compute hash keys and build the in-memory hash table.
- **Probed Table: film** (Outer Table). It is scanned sequentially (**Seq Scan on film f**), and each row's **language_id** is hashed and checked against the in-memory hash table for matches.

2. Memory Check

- **Memory Usage:** Exactly **9 kB** (as indicated in line 7: **Buckets: 1024 Batches: 1 Memory Usage: 9kB**).
- **Batch Count: Batches: 1**, indicating the entire hash table fit within memory in a single pass without spilling to disk.

3. Hypothesis (SET work_mem = '64kB');

- **Outcome:** The execution plan **remains unchanged** and continues to execute entirely in memory as a single batch (**Batches: 1**).
- **Reason:** The hash table for the **language** table only requires **9 kB** of memory. Because 9 kB is well below the allocated **64 kB** limit, the hash table still fits entirely within RAM, avoiding any multi-batch partitioning or disk spillage.

Lab 3:

Data Output		Messages	Notifications
			
Showing rows: 1 to 16		Page No: 1	of 1
QUERY PLAN			
1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=13.571..23.396 rows=4581.00 loops=1)		
2	Merge Cond: (f.film_id = i.film_id)		
3	Buffers: shared hit=86 read=26		
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.086..2.661 rows=1000.00 loo...		
5	Index Searches: 1		
6	Buffers: shared hit=83 read=1		
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=13.444..14.558 rows=4581.00 loops=1)		
8	Sort Key: i.film_id		
9	Sort Method: quicksort Memory: 300kB		
10	Buffers: shared hit=3 read=25		
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=4.218..9.365 rows=4581.00 loops=1)		
12	Buffers: shared read=25		
13	Planning:		
14	Buffers: shared hit=322 read=5 dirtied=2		
15	Planning Time: 39.090 ms		
16	Execution Time: 30.649 ms		
Total rows: 16		Query complete 00:00:00.315	CRLF Ln 13, Col 20

1. Structure: Do you see a **Sort node? Or did it use **Index Scan** to get sorted data for free?**

Both strategies are present in this execution plan:

- **For film f:** It used an **Index Scan using film_pkey on film f**. Because the Primary Key on **film_id** is a B-Tree index (which stores keys in sorted order), PostgreSQL retrieved the rows **already sorted for free** without needing a Sort node.
- **For inventory i:** There is an explicit **Sort node** (Sort Key: **i.film_id**, using **quicksort** with 300 kB of memory). Since **inventory** was read via a **Seq Scan**, PostgreSQL had to sort all 4,581 rows in memory to satisfy the sorted input requirement of the Merge Join algorithm.

2. Compare: Reset the settings (RESET ALL;) and remove the ORDER BY. Does it switch back to Hash Join? Why?

	QUERY PLAN	
	text	🔒
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=1.807..8.740 rows=4581.00 loops=1)	
2	Hash Cond: (i.film_id = f.film_id)	
3	Buffers: shared hit=113	
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.060..1.129 rows=4581.00 loo...	
5	Buffers: shared hit=25	
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=1.720..1.723 rows=1000.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB	
8	Buffers: shared hit=88	
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.030..1.065 rows=1000.00 loo...	
10	Buffers: shared hit=88	
11	Planning:	
12	Buffers: shared hit=6	
13	Planning Time: 1.039 ms	
14	Execution Time: 9.475 ms	

- **Does it switch back to Hash Join? Yes,** PostgreSQL immediately reverts to a Hash Join.
- **Why?**
 - **No Sorting Overhead:** A Merge Join requires both inputs to be sorted. Because `inventory` lacked pre-sorted data, PostgreSQL had to pay the expensive $O(N \log N)$ cost to sort 4,581 rows.
 - **Lower Cost with Hash Join:** Without the forced settings and `ORDER BY`, the Optimizer evaluates that building an in-memory Hash Table on the smaller table (`film`, 1,000 rows) and probing it with `inventory` takes linear $O(N + M)$ time. This completely avoids the sorting overhead and results in a significantly lower total execution cost.